

# Let's Talk About Package Specification

**Document:** P1313R0  
**Date:** 2018-10-07  
**Project:** ISO/IEC JTC1 SC22 WG21 Programming Language C++  
**Audience:** Study Group 15 (Tooling)  
**Author:** Matthew Woehlke ([mwoehlke.floss@gmail.com](mailto:mwoehlke.floss@gmail.com))

## Abstract

This paper explores the concept of a package specification — an important aspect of interaction between distinct software components — and recommends a possible direction for improvements in this area.

### Contents

- [Abstract](#)
- [Background](#)
- [Details of the Problem](#)
- [Objective](#)
  - [Location, Location, Location](#)
  - [What Can You Do for Me?](#)
  - [Are We Compatible?](#)
- [Historic Approaches](#)
- [A Modest Proposal](#)
- [Acknowledgments](#)

## Background

SG15 has recently been studying C++ package management. While we commend this effort, we are concerned that it may be overly focused on only one aspect — albeit an important one — of the overall problem of building a software project against external dependencies.

Given the prevalence of existing package managers with Linux-based operating system distributions, we do not feel that any single C++-oriented package manager, even should a cross-platform tool emerge, is a panacea as far as solving for external dependencies.

There are three aspects to a software project consuming an external dependency. Firstly, the dependency must be present on the system. A package manager is primarily targeted at solving this problem, and indeed this is an important problem that is worth consideration.

Secondly, the compiler and/or linker must be able to consume the dependency. This is typically accomplished by passing the name and/or location of the dependency or components thereof to the command line of the tool in question. This is largely a function of how the compiler and/or linker is invoked, which is traditionally governed by a "build tool" or "build system". Many such tools exist, with a variety of trade-offs, and given the complexity of such tools and the frequent need to support multiple languages with a single tool, we do not feel that convergence upon a single tool is a practical goal.

Lastly, there is the need to integrate the previous two points. That is, simply installing a package is insufficient; the build tool needs to know how to *locate* an installed package, and how to *use* the components which are provided by that package.

## Details of the Problem

There are a number of ways to make a software package available on a given system. These include, but are not necessarily limited to:

- Installing the package to a system location using a dedicated installer.
- Installing the package to a location (system-wide or otherwise) using a mechanism which is integrated with (and usually, to some degree, provided by) the package's build system.
- Installing the package via a dedicated general purpose package manager which is integrated with the operating system.
- Installing the package via a (possibly cross-platform) C++ package manager.
- Building the package in a user directory *without* an explicit "installation".

Because a package can come from a number of sources, we feel that relying on the package manager to integrate package usage information is a decidedly sub-optimal approach. Nor is there any need to do so; it is not difficult for a software package to include a "specification" — a file (or collection of files) describing the contents of the package and providing the necessary information for a consumer to make use of the components that the package provides.

The mechanism by which this "package specification" is generated is unimportant; it could be hand-crafted, generated by the build system, or generated by the packing tool. The key features are, first, that there exists a mechanism by which the specification may be located by a consumer. Second, that the specification can be parsed by the consumer and provides information sufficient to the consumer's needs. Finally, whether a particular package meets a consumer's needs.

## Objective

By now, we hope we have identified the problem of package *specification*, and shown how, although closely related, it differs from the problem of package *distribution*. In particular, we hope that by showing how there are, and likely always will be, multiple channels for *distribution*, we have made a case for considering the problems of specification and distribution independently.

While we will eventually consider a potential solution, our main objective is to begin a dialog on the subject of specification, and to consider how this problem might — by avoiding tight coupling with an particular build tool or package distribution mechanism — be approached in a manner that will benefit the largest number of consumers.

We will now examine the three objectives of package specification in more detail.

## Location, Location, Location

In order for a specification to be useful, a consumer must be able to *find* it. Package management (distribution) will play a crucial role here, in that a package manager should ensure that specifications are made available in a well known location. On Linux systems, this likely means taking direction from the [Filesystem Hierarchy Standard](#), while other platforms may have other existing standards, or greater latitude for creating new standards. In addition, some packages may necessarily live outside of "standard" locations. (Packages built in a user

directory that have not been "installed" are a prime example.) We expect that consumers will have a mechanism by which the user may specify additional search locations or the location of a specific package.

## What Can You Do for Me?

A specification must also inform the consumer what components are available. Consider a large product such as Qt. This product consists of a number of components (Core, GUI, Widgets, XML, etc.) which may or may not have a one-to-one correspondence with distributed packages. Thus, it is not only necessary that a package specification enumerate what components and features are available, but we can see how it would also be useful if a specification for a single "package name" can be subdivided into multiple files, so that available components may be packaged separately.

Additionally, a component may be available in different "flavors". This can span multiple axes, such as static versus shared libraries, with or without debugging utilities, or different threading support. Thus, a mechanism to allow a user to select between such alternatives is desired.

Lastly, a component needs to communicate its usage requirements to a user. This could include the locations of headers (or modules), names and locations of libraries to be linked, and other compile or link flags that must be used. However, even here we run into complications. Usage requirements are best communicated at an abstract level, such as "I require C++14" or "I require linking with pthread support". Such abstract descriptions help to avoid issues due to compiler differences, and help consumers to make more appropriate choices in the face of "conflicting" requirements.

## Are We Compatible?

Although part of this answer is implicit in a package's component set, we certainly also need to record the package version, as consumers very often depend on a particular version of a package. It is also desirable that a package specification is able to record compatibility information, so that a consumer "validated" against an older version knows if it can safely use a newer version. (Obviously, this relies upon the package providing accurate information in this respect.)

Finally, we should consider that a system may have multiple instances of a package installed. This could take the form of different releases of the same package, or code compiled for different machine architectures. Thus, our package specification should also communicate such information so that, if multiple instances of a package are available, a consumer is able to identify and select which (if any) matches its requirements.

## Historic Approaches

Many developers will be familiar with the [pkg-config](#) tool, which is one of the earliest tools created to help solve the problem we are discussing. While the package specification format used by pkg-config is more or less adequate for its original audience (GNU autotools on GNU/Linux systems), it has a number of limitations. First, a .pc file can only describe a single component, which results in much duplication across multiple such files belonging to a single conceptual product. Second, it is limited to providing tool flags, which can be problematic for build tools (such as [CMake](#)) which desire the canonical location of libraries, and makes it poorly suited for describing non-library components (such as a code generating executable). Third, its ability to specify architecture information or provide component variations is limited. Fourth, it is not well suited for non-POSIX platforms where libraries use different artifacts at link-time and run-time (such as Windows).

It is because of these issues that CMake discourages reliance on pkg-config, and eventually developed its own system of package specification. However, CMake's exported targets, while far superior in many ways (at least for CMake-using consumers), are not without their own flaws. Architecture specification is theoretically possible, but there exists no standard for doing so. Similarly, while there is some concept of "build

configuration", there is no standard, well defined mechanism for providing component variations, especially on multiple axes. Most critically, however, is that CMake uses its own (Turing complete!) language for package specification, making CMake's exported targets effectively inaccessible to any tool other than CMake itself.

This list is not exhaustive. Other build systems may provide their own tool-specific mechanisms for package specification. However, pkg-config, with all its limitations, remains the only de-facto "portable" standard of package specification.

## A Modest Proposal

It is not the primary objective of this paper to solve these problems at this time. Rather, we desire to open discussion on these issues with a long term goal of producing a new mechanism of package specification that can achieve the goals that were outlined above.

That being said, following a discussion at the 2016 Jacksonville meeting, work began on a [Common Package Specification](#) with the aim of providing a satisfactory alternative to existing mechanisms. The Common Package Specification was carefully developed based on lessons learned by CMake and has met with some positive reception already. We feel that CPS may be a viable solution, but it needs wider exposure and a functional implementation. In particular, it would benefit greatly from sponsorship willing to contribute to its further development.

## Acknowledgments

We wish to thank everyone at Kitware who has supported work on CPS and made suggestions that have furthered its development. We also wish to thank everyone who has participated in the WG21 meetings where these issues have been, and will be, discussed.